

NUOVO MONDO

Documentazione Tecnica Informatica

Python · Strutture Dati · Logica di Gioco · Salvataggio

Milani · Crotti A. · Carrara G.

INDICE

Struttura della documentazione

1

Introduzione e obiettivi del progetto

2

Analisi della consegna e requisiti

3

Architettura e organizzazione del codice

4

Strutture dati globali

5

Costanti e dizionari di configurazione

6

Funzioni principali – Fasi di gioco

7

Funzioni – Gestione eventi del viaggio

8

Funzioni – Morale, ammutinamento, scorte

9

Funzioni – Baratto, epilogo, file

10

Flusso operativo completo

11

Considerazioni tecniche e conclusioni

1. INTRODUZIONE

Descrizione generale del progetto

Descrizione del progetto

Nuovo Mondo è un gioco di simulazione scritto in Python, riadattato da un titolo pubblicato da DeAgostini sulla rivista 'Il mio computer' negli anni 1984/85.

Il giocatore gestisce una spedizione commerciale verso terre inesplorate: ingaggia un equipaggio, compra provviste e merci da barattare.

Il giocatore deve affrontare imprevisti durante il viaggio, riuscendo ad arrivare vivo nelle terre inesplorate.

Una volta arrivato avrà il compito di barattare con la popolazione locale, in modo da rientrare in patria, con alto profitto

Obiettivo della consegna

Implementare in Python il gioco seguendo le consegne,

Contesto tecnico

Linguaggio:	Python 3
Librerie:	random, json
Interfaccia:	Terminale (console)
File di output:	salvataggio.txt (JSON)
Monete iniziali:	2000
Durata base viaggio:	8 settimane
Max equipaggio:	16 persone

2. ANALISI DELLA CONSEGNA

Requisiti e funzionalità obbligatorie

Funzionalità obbligatorie richieste

Requisito	Dettaglio implementativo
Fase ingaggio flotta	Min 5 persone, almeno 1 per ruolo, max 16. 5 ruoli disponibili.
Acquisto provviste (scorte)	4 tipologie: verdura, frutta, carne, acqua. Pagamento immediato, saranno scalate ogni settimana
Acquisto merci da baratto/utilizzo	6 tipologie: medicinali, armi, sale, stoffa, coltelli, diamanti. (medicinali e armi, utilizzo)
Viaggio – 8 settimane base	6 step per settimana: evento, scorte, morale, riepilogo, ammutinamento, ricalcolo. Le settimane possono aumentare/diminuire
17+ eventi imprevisti	Ciascuno max 1 volta (albatro 3x, nessun imprevisto non ha volte Max). Logica di lista con remove().
Sistema morale	Delta morale cumulativo applicato a fine settimana a ogni membro vivo. Varia in base a situazioni come eventi o scelte decisive
Ammutinamento	Punteggio calcolato ogni fine settimana; ≥ 100 = GAME OVER. Varia in base a situazioni come eventi o scelte decisive
Baratto nel Nuovo Mondo	4 merci barattabili $\times$ 3 opzioni di scambio ciascuna. Sisceglie un solo elemento in output (da acquistare) da barattare per un'intera qta dell'elemento in input(posseduto)
Epilogo e profitto	Calcolo profitto con oscillazione casuale (x0.5, x1, x2).
Salvataggio/caricamento	File JSON (salvataggio.txt). Salva dopo ingaggio e ogni settimana.
3 esiti finali	Positivo (monete > 0), Nullo ($= 0$), Negativo (< 0) + asta nave.

3. ARCHITETTURA DEL PROGETTO

Organizzazione del codice – file unico gioco.py

File unico – gioco.py

Il progetto è contenuto in un singolo file Python (gioco.py).

COSTANTI e CONTENITORI

RUOLI, CATALOGO_CIBO, CATALOGO_MERCI, Dati fissi di configurazione del gioco.

FUNZIONI FREQUENTI

input_intero(), input_float(), si_no(), conta_vivi(),
ce_ruolo(), Min_sparo_difesa(), prova_unica()

FUNZIONI EVENTI

uomo_in_mare(), epidemia(), attacco_pirata(),
avvistamento_albatro(), avvistamento_scialuppa(), ...

FUNZIONI BARATTO/EPILOGO

baratto_sale/stoffa/coltelli/diamanti(),
Profitto(), Calcolo_spesa equipaggio(), Asta()

VARIABILI GLOBALI

stato_gioco, moltiplicatori_razioni, qta_da_perdere
Stato mutevole della partita corrente.

FUNZIONI DI FASE

fase_ingaggio(), fase_acquisto_cibo(), fase_acquisto_merci()
Gestiscono le 3 fasi preparatorie.

FUNZIONI MORALE/AMMUTINAMENTO

aggiorna_DELTA_morale(), aggiungi_mor_equip(),
ammutinamento(), ricalcolo_settimane(), Controllo_scorse()

CORPO PRINCIPALE

Loop while con gestione settimane, eventi, game_over. Stampa tramite print delle informazioni

4. STRUTTURE DATI – stato_gioco

Dizionario annidato principale – cuore dello stato di partita

stato_gioco – Dizionario annidato (dizionario di dizionari)--->

Contiene tutte le informazioni dello stato corrente della partita. Viene serializzato su file JSON per il salvataggio.

Campo	Tipo	Descrizione
monete	int/float	Monete iniziali del giocatore (2000 alla partenza)
monete_spese	float	Totale monete già spese per scorte e merci (pagate subito)
Equipaggio	dict	Dizionario di dizionari: chiave = ID membro, valore = attributi
prossimo_id	int	Contatore auto-incrementale per assegnare ID ai nuovi membri
cibo.(acqua,carne ecc....)	float	Kg di (verdura , carne,acqua e frutta) disponibili a bordo
merci.*	int	Quantità di ciascuna merce (medicinali, armi, sale, stoffa, coltelli, diamanti, perle, manufatti, spezie) (dizionario)

```
stato_gioco = {
    "monete": 2000,
    "monete_spese": 0,
    "equipaggio": {},
    "prossimo_id": 1,
    "cibo": {
        "verdura": 0.0,
        "frutta": 0.0,
        "carne": 0.0,
        "acqua": 0.0,
    },
    "merci": {
        "medicinali": 0,
        "armi": 0,
        "sale": 0,
        "stoffa": 0,
        "coltelli": 0,
        "diamanti": 0,
        "perle": 0,
        "manufatti": 0,
        "spezie": 0,
    },
}
```

Equipaggio viene aggiunto successivamente tramite lafunzione apposita

Abbiamo scelto come strutture dati i dizionari e i dizionari di dizionari per diversi motivi:

- 1) Facili dacomprendere e leggere per il programmatore
- 2) Non servono logiche di controllo indici
- 3) Urili per descrivere caratterisiche (come nell’equipaggio, vivo,paga ecc..
- 4) Accesso diretto tramite chiave
- 5) Json più facile da applicare

4. STRUTTURE DATI – equipaggio

Dizionario di dizionari – ogni chiave è l'ID numerico del membro

stato_gioco['equipaggio'] – dict{ id: {attributi} }

Campo	Tipo	Descrizione
ruolo	str	Ruolo del membro: 'cuoco','marinaio','meccanico','medico','navigatore'
paga_settimanale	int	Paga in monete/settimana (copiata da RUOLI al momento dell'ingaggio) il pagamento avverrà al rientro in patria, anche se il membro è morto
morale	int	Valore attuale del morale (parte da 100). A 0 o meno il membro muore. Varia in base ad eventi e scelte importanti
vivo	bool	True se il membro è ancora in vita, False se deceduto
ingaggiato	bool	True se ingaggiato dal giocatore (va pagato), False se recuperato in mare

crea_membro(ruolo) – struttura restituita:

```
{
  "ruolo": ruolo,
  "paga_settimanale": RUOLI[ruolo]["paga_settimanale"],
  "morale": 100,
  "vivo": True,
  "ingaggiato": True
}
```

RUOLI – costante dizionario:

cuoco	15 monete/sett.	+30 pt ammutinamento se assente
marinaio	10 monete/sett.	Ruolo base, nessun bonus speciale
meccanico	15 monete/sett.	Danni timone: +1 sett. (vs 2-4)
medico	25 monete/sett.	Cura malati epidemia (1 flac./paz.)
navigatore	20 monete/sett.	Raffiche vento: +1 sett. (vs 2-4)

5. COSTANTI E DIZIONARI DI CONFIGURAZIONE

CATALOGO_CIBO · CATALOGO_MERCI · merci_baratto · moltiplicatori_razioni

CATALOGO_CIBO – consumo settimanale per membro

Tipo	Prezzo	Consumo/sett./uomo	U.M.
verdura	0.5 monete/kg	0.5 kg/sett.	kg
frutta	1.0 monete/kg	1 kg/sett.	kg
carne	2.0 monete/kg	1 kg/sett.	kg
acqua	0.5 monete/barile	0.5 barili/sett.	barili

CATALOGO_MERCI – merci acquistabili

Tipo	Prezzo	Barattabile	U.M.
medicinali	1.0/bottiglia	No	bottiglie
armi	5.0/arma	No	armi
sale	0.5/sacco	Sì	sacchi
stoffa	2.0/telo	Sì	teli
coltelli	0.5/pezzo	Sì	pezzi
diamanti	1.0/pezzo	Sì	pezzi

merci_baratto – tassi di cambio nel Nuovo Mondo

Tipo	Sale	Stoffe	Coltelli	Diamanti	Prezzo stim.
perla	0.5 sale	5 stoffe	1 coltello	2 diamanti	2 monete
manufatto	0.5 sale	7 stoffe	3 coltelli	4 diamanti	2 monete
spezie	1 sale	3 stoffe	6 coltelli	4 diamanti	1 moneta

moltiplicatori_razioni – variabile globale

`{"verdura": 1, "frutta": 1, "carne": 1, "acqua": 1}`

Dizionario che traccia il fattore di consumo attuale per ciascuna scorta. Inizia a 1 (razione piena). Dimezzare → *0.5; Raddoppiare → *2. Usato da Sottrai_consumo_settimana() e Controllo_scorse().

qta_da_perdere = [0.5, 0.33, 0.25, 0.2]

Listache serve per eliminare delle quantità perse dagli eventi di perdita (½, ⅓, ¼, ⅕). Usata da prova_unica().

6. FUNZIONI – UTILITY E FASI PREPARATORIE

Funzioni di supporto e gestione delle fasi iniziali

Funzione	Parametri	Ritorna	Descrizione
input_intero(prompt, min, max)	str,int,int	int	Legge input intero da console. Validazione tramite min/max.
input_float(prompt, min)	str, float	float	Legge input float da console. Validazione min (default 0.0).
si_no(domanda)	str	bool	Richiede risposta 'si'/'no'. Loop finché valido. Restituisce True/False.
monete_disponibili()	–	float	Restituisce stato_gioco['monete'] - stato_gioco['monete_spese'].
crea_membro(ruolo)	str	dict	Crea e restituisce dizionario membro con morale=100, vivo=True, ingaggiato=True.
aggiungi_membro(ruolo)	str	–	Aggiunge membro a equipaggio con ID auto-incrementale (prossimo_id).
conteggio_ruoli()	–	dict	Conta quanti membri per ogni ruolo nell'equipaggio corrente.
totale_equipaggio()	–	int	len(stato_gioco['equipaggio']): numero totale di membri (vivi+morti).
paga_stimata(settimane=8)	int	float	Somma delle paghe_settimanali × settimane per tutti i membri.
conta_vivi(equipaggio)	dict	int	Conta i membri con vivo==True. Usata ovunque come callback.
ce_ruolo(equipaggio, ruolo)	dict,str	bool	True se esiste almeno un membro vivo con quel ruolo.
Min_sparo_difesa(equipaggio, merci, fn_vivi)	dict,dict,fn	int	min(vivi, armi): tentativi disponibili per sparare o difendersi.

Scelta dell'evento e eliminazione di esso

Funzioni di supporto e gestione delle fasi iniziali

C'è una lista che contiene tutti interi (ogni intero è associato ad un evento come visto nella slide precedente), un intero Corrisponde ad un ID

```
li_eventi=[0,1,2,3,4,5,6,7,8,9,10,11,90,91,12,13,14,15,16,17,18]
```

Ad ogni iterazione del while la cui condizione di fine è:

```
while conta_set<=settimane and not game_over and punti_ammutinamento<100:
```

Viene estratto un ID dalla lista tramite un random.choice (che appunto scegli l'id)

```
n_evento=random.choice(li_eventi)
```

Dopo la chiamata della funzione che riguarda l'evento pescato, verrà rimosso dalla lista l'ID dell'evento appena capitato (tranne evento 18)

Ciò avviene tramite

```
li_eventi.remove(5)
```

Esempio di funzione che corrisponde ad un evento

```
elif n_evento==5:
    stato_gioco["cibo"],qta_pescata=tempesta_miracolosa(stato_gioco["cibo"],"carne")
    print(f"--- EVENTO: PESCA MIRACOLOSA ---\nDurante la settimana di quiete l'equipaggio ha pescato {qta_pescata} kg di CARNE.")
    li_eventi.remove(5)
```

GESTIONE DELLE SETTIMANE DI VIAGGIO

Variabili, loop principale e condizioni di modifica della durata

Variabili di controllo

SETTIMANE = 8

Costante intera. Indica la durata originale del viaggio. Non viene mai modificata.

settimane = 8

Variabile mutabile. Viene incrementata/decrementata dagli eventi.

sett_vecchie = 8

Settimane precedenti a quella attuale. Usata per calcolare `differenza_settimane = settimane - sett_vecchie`. -> utile per ammutinamento

conta_set = 1

Contatore della settimana. Incrementa di 1 a ogni iterazione.

Condizione del loop principale

```
while conta_set <= settimane and not game_over and punti_ammutinamento < 100:
```

Cause di variazione delle settimane

Venti favorevoli

settimane -= 1

Viaggio accorciato di 1 settimana. Evento casuale.

Danni al timone

settimane += 1
o += randint(2,4)

Con meccanico: +1. Senza meccanico: +2÷4.

Raffiche di vento

settimane += 1
o += randint(2,4)

Con navigatore: +1. Senza navigatore: +2÷4.

Avvistamento isola

settimane += randint(1,2)

Solo se il giocatore decide di approdare.

Basso morale (ricalcolo)

settimane += 1

Se metà+ dei vivi ha morale ≤ 30 a fine settimana.

Viaggio ritorno

+1 o +2 sett.

1 sett. con navigatore vivo, 2 senza. +1 se albatro ucciso.

6. FUNZIONI – FASI DI GIOCO

fase_ingaggio · fase_acquisto_cibo · fase_acquisto_merci

FASE 1 – INGAGGIO

Funzioni coinvolte:

fase_ingaggio()
turno_ingaggio()
stampa_stato equipaggio()
riepilogo_ingaggio()

Il giocatore compone l'equipaggio.

- Min 5 persone, almeno 1 per ruolo
- Max 16 persone
- Loop *turno_ingaggio()* ricorsivo
- La paga NON viene detratta subito
- Salva stato dopo il completamento

FASE 2 – ACQUISTO CIBO

Funzioni coinvolte:

fase_acquisto_cibo()
turno_cibo(indice)
stampa_stato_cibo()

Il giocatore acquista le provviste.

- Navigazione ricorsiva per indice
- Mostra quantità consigliata per 8 sett.
- Controllo monete disponibili
- *monete_spese* += costo immediato
- Il giocatore può comprare 0 di ogni tipo

FASE 3 – ACQUISTO MERCI

Funzioni coinvolte:

fase_acquisto_merci()
turno_merci(indice)
stampa_stato_merci()

Il giocatore acquista merci da barattare.

- Stessa logica ricorsiva di *turno_cibo*
- Acquisto in interi (*input_intero*)
- *monete_spese* += costo immediato
- Non c'è obbligo di acquistare

7. FUNZIONI – GESTIONE EVENTI DEL VIAGGIO

Meccanismo eventi: lista li_eventi con ID numerici, random.choice() ogni settimana, remove() dopo uso.

Sistema di eventi randomici con lista di ID univoci

li_eventi = [0,1,2,3,4,5,6,7,8,9,10,11,90,91,12,13,14,15,16,17,18]

ID	Nome evento	Funzione	Effetto
0	Uomo in mare	uomo_in_mare(equipaggio, delta)	Sceglie membro vivo casuale, lo imposta vivo=False, delta_morale -10
1-4	Scorte in mare	prova_unica(merci/cibo, tipo, qta)	Toglie una frazione casuale [0.5,0.33,0.25,0.2] di verdura/frutta/carne/acqua
5	Pesca miracolosa	tempesta_miracolosa(cibo,'carne')	Aggiunge casualmente un valore tra 11 e 20 kg di carne
6	Tempesta miracolosa	tempesta_miracolosa(cibo,'acqua')	Aggiunge casualmente un valore tra 11 e 20barili d'acqua
7	Venti favorevoli	vento_favorevole(sett,equip,delta)	settimane -1, delta_morale +randint(5,15)
8-10	Cattivo tempo/Ondata/Ratti	prova_unica(merci,tipo,qta)	Perdita frazionaria di medicinali/armi/stoffe
11,90,91	Avvistamento albatro	avvistamento_albatro(...)	Se armi $\geq$ 1 chiede se sparare. Colpo 50%. Se ucciso aggiunge dai 10ai 15 (casualmente) kg carne. Armi usate rimosse.
12	Avvistamento scialuppa	avvistamento_scialuppa(...)	Se accettati: +4 naufraghi (ingaggiato=False), morale 25-75, merci +randint(10,20) tutte
13	Raffiche di vento	raffiche_di_vento(equip,sett,ruolo)	Con navigatore: +1 sett. Senza: +randint(2,4) sett.
14	Epidemia	epidemia(equip,merci,ce_med,delta)	70% prob. malattia per ogni non-medico vivo. Medico+medicinale = cura. Altrimenti morte. Si consuma la bottiglia di medicinale
15	Attacco pirata	attacco_pirata(equip,vivi,merci,min_s_p,delta)	n_pirati=randint(3,10). uomini_persi=min(pirati-difensori, vivi). Le armi utilizzate vengono tolte
16	Avvistamento isola	avvistamento_isola(dec,albatro,merci)	50% abitata. 50% ostile. Se amichevole: merci la quantità delle merci donate varia da 5 a 20 o da 20° 40 se nessun albatro ucciso.
17	Danni al timone	–	Il viaggio si allunga di 1 settimana se è presente il meccanico sennò si allunga di 2-4 settimane
18	Nessun imprevisto (permanente)	–	Nessun effetto. NON rimosso dalla lista (può capitare più volte).

8. FUNZIONI – MORALE, SCORTE E AMMUTINAMENTO

Sistema di delta morale e controllo settimanale delle provviste

Sistema Morale

`aggiorna_DELTA_morale(delta, qta)`

delta_mor += qta – aggiorna il delta accumulato

`aggiungi_mor equip(delta)`

Applica delta a ogni membro vivo. Se morale ≤ 0 → vivo=False

`Controllo_scorse(...)`

Detrae consumo settimanale, valuta stato scorte, modifica delta

`Stato_cibo(cibo, fabbisogno)`

Ritorna 'esaurita'/'insufficiente'/'abbondante'/'normale'

`Chiedi_modifica_razione(cibo, stato)`

Chiede al giocatore se dimezzare/raddoppiare la razione

`Aggiorna_moltiplicatore_e_morale(...)`

Modifica moltiplicatore e delta morale in base alla scelta

Variazioni delta morale:

+5 raddoppio razione | -5 dimezzamento | -10 scorta esaurita | +5÷+15 vento favorevole | -5 epidemia | -5/-10 eventi letali

Ammutinamento

`ammutinamento(dimezzato, ce_ruolo, equip, albatro_colpito, conta_vivi, diff_sett)`

Condizione	Variazione
Razione dimezzata (≥1 tipo)	+30 punti
Più di 12 uomini vivi	+30 punti
Albatro ucciso (colpito==True)	+30 punti
Albatro avvistato non ucciso (==False)	-20 punti
Cuoco assente	+30 punti
Ogni sett. extra rispetto alle 8 base	+10 punti/sett.

Esiti:

1–99 → avviso al giocatore
≥100 → GAME OVER ammutinamento
≤0 → nessuna segnalazione

`ricalcolo_settimane(sett, equip, conta_vivi)`

Se metà o più dei vivi ha morale ≤ 30 → settimane += 1

9. FUNZIONI – BARATTO, EPILOGO E FILE

Arrivo nel Nuovo Mondo · Tradimento · Profitto · Salvataggio

Funzioni Baratto

`baratto_sale/stoffa/coltelli/diamanti(m_bar, stato)`

Per ogni merce calcola 3 offerte (int division). Il giocatore sceglie 1. La merce scende a 0 e si accumulano perle/manufatti/spezie.

`Resoconto_baratto(merci_baratto, stato)`

Stampa riepilogo di perle, manufatti, spezie ottenuti con valore stimato.

`Intro_tradimento(m_bar, stato)`

Mostra offerta del rivale (30 perle/arma). Chiede s/n al giocatore.

`Tradimento(stato, albatro)`

Se albatro==True: scoperto=True. Se albatro==None: scoperto=random(1,2)==1. Aggiunge perle, azzera armi.

`Ricompensa(albatro)`

Se albatro ucciso: randint(5,20) perle. Altrimenti: randint(30,50) perle. Premio per non aver tradito.

`calcolo_settimane_e_rifornimento(stato, albatro, catalogo)`

Rifornisce scorte per 3 sett. Calcola settimane ritorno: 1 con nav., 2 senza, +1 se albatro ucciso.

Funzioni Epilogo e File

`Profitto(stato, merci_baratto)`

`oscillazione = random.choice([0.5,1,2])`
`profitto = perle×prezzo×osc + manufatti×prezzo×osc + spezie×prezzo×osc`

`Calcolo_spesa equipaggio(stato, sett)`

Somma `pagaxsettimane` solo per i membri con `ingaggiato==True` (esclude naufraghi).

`Stampa_situazione_economica(profitto, stato, spesa)`

Aggiunge profitto a monete, sottrae spesa equipaggio. Stampa situazione finale.

`Asta(stato)`

Due liste di offerte: `valori_temp` (usabili 1 volta ciascuna) e `valori=[50,300,400,450]` (infinite). Loop fino ad accettazione.

`Good_ending / Neutral_ending / Bad_ending()`

3 epiloghi in base al segno di `stato['monete']` a fine gioco.

`Salva(stato) / Carica()`

`json.dump()` e `json.load()` su `FILE_SALVATAGGIO='salvataggio.txt'`. Salvataggio dopo ingaggio e ogni fine settimana.

11. CONSIDERAZIONI TECNICHE E CONCLUSIONI

Scelte implementative · Conformità alla consegna

Scelte implementative

Dizionari annidati

stato_gioco è un dizionario di dizionari: accesso diretto per chiave, struttura leggibile, facilmente utilizzabile in JSON.

Ricorsione per fasi

turno_ingaggio(), turno_cibo(), turno_merci() si richiamano ricorsivamente invece di usare while loop.

Lista eventi con remove()

li_eventi gestisce unicità degli eventi: remove() dopo uso garantisce che ogni evento capiti max 1 volta (eccetto albatro con 3 ID e nessun imprevisto ID 18).

delta_morale accumulatore

Il morale non viene modificato direttamente ogni evento, ma accumulato in delta_morale e applicato una volta a fine settimana.

JSON per salvataggio

json.dump/load gestisce strutture annidate complesse. Il file salvataggio.txt viene sovrascritto dopo ingaggio e ogni fine settimana. Carica viene deciso appena si apre il gioco

Unico file Python

Tutto il codice è in gioco.py: costanti, funzioni, corpo principale. Nessun modulo aggiuntivo oltre a random e json.

colpito = None

Variabile globale inizializzata a None (albatro mai avvistato). Usata per discriminare 3 stati: None/True/False.

GESTIONE DELLE SETTIMANE DI VIAGGIO

Variabili, loop principale e condizioni di modifica della durata

Variabili di controllo

SETTIMANE = 8

Costante intera. Indica la durata originale del viaggio. Non viene mai modificata.

settimane = 8

Variabile mutabile. Viene incrementata/decrementata dagli eventi. È il tetto del loop.

sett_vecchie = 8

Snapshot iniziale. Usata per calcolare $\text{differenza_settimane} = \text{settimane} - \text{sett_vecchie}$.

conta_set = 1

Contatore corrente della settimana. Incrementa di 1 a ogni iterazione del loop.

Condizione del loop principale

```
while conta_set <= settimane and not game_over and punti_ammutinamento < 100:
```

Cause di variazione delle settimane

Venti favorevoli

settimane -= 1

Viaggio accorciato di 1 settimana. Evento casuale.

Danni al timone

settimane += 1
o += randint(2,4)

Con meccanico: +1. Senza meccanico: +2÷4.

Raffiche di vento

settimane += 1
o += randint(2,4)

Con navigatore: +1. Senza navigatore: +2÷4.

Avvistamento isola

settimane += randint(1,2)

Solo se il giocatore decide di approdare.

Basso morale (ricalcolo)

settimane += 1

Se metà+ dei vivi ha morale ≤ 30 a fine settimana.

Viaggio ritorno

+1 o +2 sett.

1 sett. con navigatore vivo, 2 senza. +1 se albatro ucciso.

I RUOLI DELL'EQUIPAGGIO

5 ruoli ingaggiabili · costi settimanali · funzioni speciali nel gioco

CUOCO

15 monete/sett.

Ruolo base per la qualità del cibo a bordo.

- Se assente: +30 punti ammutinamento ogni fine settimana
- Nessun ruolo speciale negli eventi imprevisti
- Essenziale per mantenere basso il rischio di ammutinamento
- La sua assenza pesa quanto una razione dimezzata o un albatro ucciso

MARINAIO

10 monete/sett.

Ruolo generico. Il meno costoso dell'equipaggio.

- Nessuna funzione speciale negli eventi
- Utile per aumentare il numero di difensori nell'attacco pirata
- Utile per aumentare i tentativi di sparo all'albatro
- Il costo minore lo rende ideale per 'riempire' l'equipaggio

MECCANICO

15 monete/sett.

Specialista nella riparazione della nave.

- Evento Danni al timone: con meccanico +1 settimana (senza: +2÷4)
- Verificato tramite `ce_ruolo(equipaggio, 'meccanico')`
- Può fare la differenza in termini di settimane extra e costi paga
- Stessa paga del cuoco ma con effetto su durata viaggio

MEDICO

25 monete/sett.

Il più costoso. Cruciale durante l'epidemia.

- Evento Epidemia: cura i malati consumando 1 medicinale/paziente
- Senza medico: i malati muoiono quasi certamente (70% prob.)
- I medici stessi non contraggono l'epidemia (esclusi dal loop)
- Verificato tramite `ce_ruolo(equipaggio, 'medico')`

NAVIGATORE

20 monete/sett.

Guida la rotta. Riduce i tempi in caso di imprevisti.

- Evento Raffiche di vento: con navigatore +1 sett. (senza: +2÷4)
- Viaggio di ritorno: 1 settimana con navigatore, 2 settimane senza
- Il suo valore cresce al crescere della durata del viaggio
- Verificato tramite `ce_ruolo(equipaggio, 'navigatore')`

Regola ingaggio: minimo 5 persone, almeno 1 per ruolo, massimo 16 totali. I naufraghi recuperati (`ingaggiato=False`) non vengono pagati a fine viaggio.

DOVE CI SIAMO DOCUMENTATI CON L'AI

Diversi utilizzi dell'IA

Risolvere conflitti GIT

15 monete/sett.

Soprattutto nella fase merge

Formattare meglio il testo delle funzioni

Usato per fare menù per rendere il tutto più leggibile

Creazione grafica, tabelle della documentazione

Queste grafiche sono state fatte con Claude perché la presentazione fatta a mano era di difficile comprensione e di livello inferiore. Alcune tabelle sono state fatte con IA

Il contenuto è nella presentazione seguente:

[Nuovo Mondo - Presentazione](#)

Risoluzioni Errori ed eccezioni

Soprattutto dopo la merge vi erano tante eccezioni, tra esse alcune non siamo riusciti a risolverle da soli

Aiuto riscrittura migliorata

Alcuni print sono stati migliorati dall'AI, come le esclamazioni. Pre il codice non è stata usata tanto

- Veniva usata nella fase di progettazione per risolvere eccezioni difficili da risolvere, soprattutto nella fase finale nei nostri branch

La comunicazione non è stata utilizzata tanto con gli altri gruppi, ne abbiamo fatto ricorso principalmente con il gruppo di Bettinelli, Damian, Brattico, perché ci siamo trovati a programmare insieme qualche volta

CONCLUSIONI

Il gioco implementa fedelmente tutte le specifiche della consegna:
fase di ingaggio, acquisto scorte/merci, viaggio con 17+ eventi, baratto, tradimento, epilogo.

Le strutture dati scelte (dizionari annidati) consentono accesso diretto per chiave ed una serializzazione JSON semplice e affidabile per il sistema di salvataggio.

Il sistema di delta_morale garantisce un aggiornamento coerente e cumulativo del morale dell'equipaggio a fine settimana.

La lista li_eventi con remove() assicura la non-ripetibilità degli eventi, rispettando il vincolo della consegna (eccetto albatro ID 11/90/91 e nessun imprevisto ID 18).

L'uso dell'IA è stato impiegato come supporto alla stesura della documentazione tecnica e principalmente per correzione errori e ordine.